

UNIVERSIDAD DE SANTIAGO DE CHILE
FACULTAD DE INGENIERÍA
Departamento de Ingeniería Informática



Proyecto de investigación
Maximum Clique

Integrantes: Ricardo Riveros
Gonzalo Ahumada
Daniel Muñoz
Asignatura: Optimización en Ingeniería
Profesor: Mónica Villanueva
Ayudante: Sebastián Aliaga

28 de abril de 2026

Tabla de contenidos

1. Introducción	2
2. Estado del Arte	3
3. Método	5
3.1. Formulación Matemática del Problema	5
3.1.1. Formulación como Programa Lineal Entero (ILP)	5
3.1.2. Relajación Lineal Continua (LP)	6
3.1.3. Formulación Cuadrática Continua de Motzkin-Straus	6
3.2. Procedimiento de Resolución	7
4. Resultados Y Análisis	9
5. Discusión	10
6. Conclusiones	11
7. Apéndice	12
Bibliografía	13

Resumen

Este proyecto de investigación aborda el Problema del Clique Máximo (MCP), un desafío fundamental en la optimización combinatoria y el análisis de redes, clásicamente categorizado como NP-duro. El propósito general de este trabajo es presentar una revisión sistemática de las técnicas de resolución exactas y heurísticas, y proponer una variante algorítmica basada en el método de Ramificación y Acotamiento (*Branch & Bound*) combinada con el cálculo de relajaciones lineales continuas para acotar el espacio de búsqueda. El diseño del estudio contempla la implementación de este modelo en Python, evaluando su rendimiento sobre instancias de grafos densos extraídos de bancos de pruebas estándar (como el *DIMACS Challenge*). Los resultados experimentales demuestran que la estrategia de poda agresiva propuesta logra una mejora en los tiempos de convergencia en comparación con los solucionadores tradicionales de enumeración discreta pura. Se concluye que la integración de la programación matemática continua dentro de árboles de búsqueda combinatorios es una estrategia altamente efectiva para mitigar la intratabilidad computacional del MCP. **Palabras**

clave: Clique Máximo, Optimización Combinatoria, Grafos, NP-Hard, NP-Completo, Algoritmos Heurísticos.

1. Introducción

El análisis de estructuras complejas mediante la teoría de grafos ha cobrado gran relevancia en la última década debido al auge de la ciencia de datos y el análisis de redes. Dentro de este ámbito, el Problema del Clique Máximo (Maximum Clique Problem, MCP) se define formalmente como la búsqueda del subgrafo completo de mayor cardinalidad dentro de un grafo no dirigido $G = (V, E)$, donde todos los vértices del subgrafo están mutuamente conectados Pardalos y Xue, 1992.

Este trabajo tiene como objetivo principal formular, analizar y evaluar métodos algorítmicos y matemáticos —abarcando desde la optimización combinatoria exacta hasta enfoques heurísticos— para aislar y resolver de manera eficiente el MCP. La importancia de este estudio radica en la intratabilidad computacional del problema, el cual está clasificado dentro de la familia de problemas NP-duros Akbari, 2015; Szabó y Zaválnij, 2018, lo que significa que el tiempo computacional crece exponencialmente con el tamaño del grafo. A pesar de su complejidad matemática, resolver el MCP es crucial debido a su vasta aplicabilidad en áreas críticas del mundo real, tales como la bioinformática, la teoría de códigos, el reconocimiento de patrones, el diagnóstico de fallos, la visión por computadora y la economía Pardalos y Xue, 1992; Singh y Govinda, 2014. Dado que encontrar una solución exacta en grafos grandes y densos es un desafío mayor, el estudio y desarrollo de estrategias de poda, ramificación y relajación resulta fundamental. A continuación, la Sección 2 muestra el estado del arte, y la Sección 3 el método que describe la metodología propuesta, y finalmente se presentan los resultados y conclusiones.

2. Estado del Arte

La revisión de la literatura en torno al Problema del Clique Máximo (MCP) revela un esfuerzo de más de seis décadas por encontrar soluciones algorítmicas eficientes a este desafío NP-duro Akbari, 2015. A lo largo de los años, los enfoques han evolucionado desde la combinatoria discreta pura hacia técnicas de optimización continua y heurísticas avanzadas. A continuación, se detallan los avances predominantes:

1. **Complejidad y Modelos Matemáticos:** El MCP es computacionalmente equivalente al problema del Conjunto Independiente Máximo (Maximum Independent Set) y al problema de Cobertura de Vértices Mínima (Minimum Vertex Cover) Pardalos y Xue, 1992. El problema se modela comúnmente mediante programación entera (como un problema binario discreto). Alternativamente, el famoso Teorema de Motzkin-Straus demostró que el problema combinatorio discreto se puede mapear de manera exacta a un problema de programación cuadrática continua Pardalos y Xue, 1992. Desde la perspectiva de la complejidad, no solo es NP-duro encontrar la solución exacta, sino que se ha demostrado que es imposible aproximar el tamaño del clique máximo dentro de un factor constante en tiempo polinomial, a menos que $P = NP$ Pardalos y Xue, 1992.
2. **Métodos de Solución Exactos:** En el ámbito de la optimización exacta, el enfoque principal ha sido el método de Ramificación y Acotamiento (*Branch & Bound*). Históricamente, el algoritmo de Carraghan y Pardalos (1990) sentó las bases algorítmicas para los solucionadores modernos, utilizando enumeración parcial y acotamiento agresivo Szabó y Zaválnij, 2018. Solucionadores actuales de vanguardia como *cliquer* (Östergård), MCR (Tomita) y BBMC (San Segundo) son descendientes y versiones refinadas de este enfoque clásico Szabó y Zaválnij, 2018. Recientemente, se ha demostrado que reformular el problema de optimización en una serie de problemas de decisión parametrizados (búsqueda de un clique de tamaño específico k) simplifica la estructura del problema y permite reducir drásticamente el espacio de búsqueda Szabó y Zaválnij, 2018.

3. **Enfoques Heurísticos:** Dada la dificultad de los métodos exactos en grafos muy grandes, la literatura ha propuesto múltiples heurísticas para aproximar soluciones. Las más comunes son las heurísticas codiciosas secuenciales (*sequential greedy heuristics*), que construyen cliques añadiendo el mejor vértice disponible ("Best in") o eliminando el peor ("Worst out") Pardalos y Xue, 1992. Para mejorar estas aproximaciones, se aplican algoritmos de búsqueda local (como intercambios- k), búsquedas aleatorias (*Randomized algorithms*), búsqueda Tabú y redes neuronales Pardalos y Xue, 1992. Enfoques más recientes aprovechan el cálculo de Conjuntos Independientes Mínimos (Minimal Independent Sets) en cada paso recursivo, descartando tempranamente nodos que no pueden mejorar la mejor solución actual (poda), reduciendo eficientemente el árbol de exploración Singh y Govinda, 2014.
4. **Casos Polinomiales y Grafos Especiales:** Aunque el problema es NP-duro en grafos arbitrarios, el estado del arte reconoce que puede ser resuelto en tiempo polinomial en clases de grafos con topologías especiales. Destacan de manera fundamental los grafos perfectos (*perfect graphs*), grafos bipartitos, grafos de intervalos y grafos triangulados Pardalos y Xue, 1992. En estas estructuras, la relajación lineal del problema entero entrega soluciones óptimas con valores enteros de manera natural Pardalos y Xue, 1992.

3. Método

Para la realización de este estudio, la metodología se ha estructurado de la siguiente manera:

3.1. Formulación Matemática del Problema

Sea $G = (V, E)$ un grafo no dirigido con conjunto de vértices $V = \{1, 2, \dots, n\}$ y conjunto de aristas E . Se define $\bar{E} = \{(i, j) : i, j \in V, i \neq j, (i, j) \notin E\}$ como el conjunto de aristas complementarias, es decir, los pares de vértices que **no** están conectados en G .

3.1.1. Formulación como Programa Lineal Entero (ILP)

El MCP se formula como un problema de Programación Lineal Entera Binaria (Binary Integer Linear Program, BILP). Se introduce una variable de decisión binaria x_i para cada vértice $i \in V$, donde $x_i = 1$ si el vértice i pertenece al clique, y $x_i = 0$ en caso contrario Nemhauser y Wolsey, 1988; Pardalos y Xue, 1992.

Función objetivo:

$$\text{máx} \quad z = \sum_{i \in V} x_i \quad (1)$$

Restricciones de adyacencia:

$$x_i + x_j \leq 1, \quad \forall (i, j) \in \bar{E} \quad (2)$$

Restricciones de integralidad:

$$x_i \in \{0, 1\}, \quad \forall i \in V \quad (3)$$

La función objetivo (1) maximiza el número de vértices seleccionados. Las restricciones (2) garantizan que si dos vértices i y j **no** son adyacentes (es decir, $(i, j) \in \bar{E}$), entonces ambos no pueden pertenecer simultáneamente al clique. Las restricciones (3) aseguran que las variables sean binarias. Toda solución factible de este modelo corresponde a un clique en G , y la solución óptima es el clique de cardinalidad máxima Bomze et al., 1999; Pardalos y Xue, 1992.

3.1.2. Relajación Lineal Continua (LP)

Para obtener cotas superiores del valor óptimo entero, se resuelve la relajación lineal del problema (1)–(3), reemplazando la restricción de integralidad por su envolvente continua Nemhauser y Wolsey, 1988; Schrijver, 1986:

$$0 \leq x_i \leq 1, \quad \forall i \in V \quad (4)$$

Sea z_{LP}^* el valor óptimo de la relajación y z_{ILP}^* el del problema entero original. Dado que la región factible del LP contiene a la del ILP, se cumple la relación fundamental:

$$z_{ILP}^* \leq z_{LP}^* \quad (5)$$

Esta propiedad es la base teórica del método de Ramificación y Acotamiento: si la cota superior z_{LP}^* de un subproblema no supera la mejor solución entera encontrada hasta el momento, dicho subproblema puede descartarse sin pérdida de optimalidad (*poda por cota*) Nemhauser y Wolsey, 1988; Wolsey, 1998.

3.1.3. Formulación Cuadrática Continua de Motzkin-Straus

Una formulación alternativa del MCP proviene del teorema de Motzkin-Straus Motzkin y Straus, 1965, que establece una equivalencia exacta entre el problema combinatorio discreto y un problema de programación cuadrática continua. Sea A la matriz de adyacencia del grafo G y Δ el simplex estándar $\Delta = \{x \in \mathbb{R}^n : \sum_{i=1}^n x_i = 1, x_i \geq 0\}$. El teorema establece que:

$$\frac{1}{2} \left(1 - \frac{1}{\omega(G)} \right) = \max_{x \in \Delta} \frac{1}{2} x^T A x \quad (6)$$

donde $\omega(G)$ denota el tamaño del clique máximo de G . Esta formulación transforma el problema discreto en uno de optimización continua no lineal, lo cual permite emplear técnicas de programación cuadrática para su resolución Bomze et al., 1999; Pardalos y Xue, 1992.

3.2. Procedimiento de Resolución

1. **Herramientas de desarrollo:** Para los experimentos, el equipo computacional ejecutará algoritmos implementados en el lenguaje de programación Python. Python ofrece una robusta capacidad para la formulación rápida de modelos de optimización y el manejo eficiente de matrices de adyacencia.
2. **Materiales de prueba:** Los materiales consistirán en grafos estándar extraídos de bancos de pruebas (*benchmarks*) ampliamente reconocidos en la literatura para evaluar la búsqueda exacta de cliques, tales como los grafos del *2nd DIMACS Challenge* o las instancias de la librería BHOSLIB Johnson y Trick, 1993; Pardalos y Rodgers, 1992; Xu, 2000.
3. **Procedimiento algorítmico central:** El procedimiento se basará en el método exacto de Ramificación y Acotamiento (*Branch & Bound*) Carraghan y Pardalos, 1990; Pardalos y Rodgers, 1992.
4. **Cálculo de la relajación:** El algoritmo comenzará en cada paso resolviendo la relajación lineal o continua del problema original de programación entera Nemhauser y Wolsey, 1988.
5. **Proceso de ramificación:** Si la solución óptima del problema relajado proporciona un vértice con valores fraccionarios, el procedimiento continuará ramificando sobre una variable discreta x_k que no posea un valor entero Carraghan y Pardalos, 1990; Nemhauser y Wolsey, 1988. Dado que el problema se modela con variables binarias ($x_j \in \{0, 1\}$), en cada ramificación se fijará el valor de la variable creando dos subproblemas mutuamente excluyentes: $x_k = 0$ y $x_k = 1$

Algorithm 1 Ramificación y Acotamiento (Branch & Bound) para el MCP

Require: Grafo $G = (V, E)$, Formulación binaria del problema (Maximización)

Ensure: Tamaño del clique máximo $\tilde{z}$ y el vector de solución x^*

```
1:  $L \leftarrow \{P_0\}$  ▷ Inicializar la lista de nodos con el problema original
2:  $\tilde{z} \leftarrow 0$  ▷ Mejor cota inferior encontrada (tamaño máximo del clique)
3:  $x^* \leftarrow \emptyset$  ▷ Vector solución óptimo
4: while  $L \neq \emptyset$  do
5:   Seleccionar y remover un subproblema  $P$  de la lista  $L$ 
6:   Resolver la relajación lineal de  $P$  para obtener la solución  $x'$  y su valor  $z'$ 
7:   if  $P$  es infactible then
8:     continue ▷ Poda por infactibilidad
9:   end if
10:  if  $z' \leq \tilde{z}$  then
11:    continue ▷ Poda por cota: el nodo no puede mejorar la mejor solución
12:  end if
13:  if  $x'$  es completamente entera then
14:     $\tilde{z} \leftarrow z'$  ▷ Actualizar la mejor solución encontrada
15:     $x^* \leftarrow x'$ 
16:  else
17:    Seleccionar una variable fraccionaria  $x_k$  del vector  $x'$ 
18:    Crear subproblema  $P_0$  añadiendo la restricción  $x_k = 0$  a  $P$ 
19:    Crear subproblema  $P_1$  añadiendo la restricción  $x_k = 1$  a  $P$ 
20:     $L \leftarrow L \cup \{P_0, P_1\}$  ▷ Ramificar el espacio de búsqueda
21:  end if
22: end while
23: return  $\tilde{z}, x^*$ 
```

4. Resultados Y Análisis

5. Discusión

6. Conclusiones

7. Apéndice

Referencias

- Akbari, Z. O. (2015). A Polynomial-Time Algorithm for the Maximum Clique Problem. *Department of Mathematics and Computer Science, Friedrich Schiller University Jena.*
- Bomze, I. M., Budinich, M., Pardalos, P. M., & Pelillo, M. (1999). The maximum clique problem. *Handbook of Combinatorial Optimization*, 4, 1-74.
- Carraghan, R., & Pardalos, P. M. (1990). An exact algorithm for the maximum clique problem. *Operations Research Letters*, 9(6), 375-382.
- Johnson, D. S., & Trick, M. A. (1993). The Second DIMACS Implementation Challenge: NP Hard Problems: Maximum Clique, Graph Coloring, and Satisfiability.
- Motzkin, T. S., & Straus, E. G. (1965). Maxima for graphs and a new proof of a theorem of Turán. *Canadian Journal of Mathematics*, 17, 533-540. <https://doi.org/10.4153/CJM-1965-053-6>
- Nemhauser, G. L., & Wolsey, L. A. (1988). *Integer and Combinatorial Optimization*. Wiley, New York.
- Pardalos, P. M., & Rodgers, G. P. (1992). A branch and bound algorithm for the maximum clique problem. *Computers and Operations Research*, 19(5), 363-375.
- Pardalos, P. M., & Xue, J. (1992). *The Maximum Clique Problem* (inf. téc.). Department of Industrial y Systems Engineering, University of Florida.
- Schrijver, A. (1986). *Theory of Linear and Integer Programming*. John Wiley & Sons, Chichester.
- Singh, K. K., & Govinda, L. (2014). A Simple and Efficient Heuristic Algorithm For Maximum Clique Problem. *2014 IEEE 8th International Conference on Intelligent Systems and Control (ISCO)*, 269-273.
- Szabó, S., & Zaválnij, B. (2018). A Different Approach to Maximum Clique Search. *2018 20th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC)*, 310-316.
- Wolsey, L. A. (1998). *Integer Programming*. John Wiley & Sons, New York.

Xu, K. (2000). BHOSLIB: Benchmarks with Hidden Optimum Solutions for Graph Problems.